Autonomous Agentic Analytics Platform

From Feature Spec to Business Insight in Seconds

Atlys **PrismCH** unifies Instrumentation, Context Maintenance, and Server-Side ClickHouse Analytics into an autonomous, self-healing 3–agent ecosystem.

INSTRUMENTATION AGENT

Auto DDL & Data Ingest

Zero-data-loss table widening

CONTEXT AGENT

Living Metadata & Audit

ClickHouse-resident versioning

ANALYTICS AGENT

Server-Push Analytics

Deep "Why" root-cause synthesis

THE SCALE & THE FRICTION

700k+ Annual Visas, 120+ Destinations, Constant Product Velocity

⚠ The Analytics Bottleneck Today

Every new feature launch requires writing a tracking PRD, designing ClickHouse schemas weeks later, manually updating context docs, and writing complex SQL queries.

The 3 Breaking Points:

- ✕ **Context Drift:** Business context lives in stale markdown files while DB schemas evolve separately.
- ✕ **Suboptimal Schemas:** Default IDs and poor sorting keys lead to high latency and missed compression benefits.
- ✕ **LLM Context Burning:** Naive agents fetch raw event rows into LLM prompts, exploding token costs and crashing on big data.

THE PRISMCH OPPORTUNITY

Collapsing the Spec-to-Insight Loop

Instead of manual handoffs between Product, Engineering, and Analytics, PrismCH deploys an autonomous multi-agent pipeline that ingests raw specs and delivers actionable product insights in seconds.

100%
Server-Side Query Pushdown

< 10s
Spec Ingestion & Report Cycle

ARCHITECTURE OVERVIEW

Tri-Agent Cooperative Ecosystem

Agent 1

Instrumentation Agent

Parses `spec.md` & event stream profiles. Generates & executes optimal DDL.

- LowCardinality & Unsigned Types
- Bounded Repair Loop
- Idempotent ALTER TABLE ADD COLUMN

Agent 2

Context Agent

Maintains immutable context snapshot store inside ClickHouse tables.

- Live ClickHouse Schema Sync
- Contradiction & Gap Detection
- Automatic Cascade to Analytics

Agent 3

Analytics Agent

Queries ClickHouse via strict safety rules & synthesizes deep root causes.

- windowFunnel & uniqCombined
- Server-Enforced readonly=1
- Slices across 4 primary dims

Core Layer: `ClickHouse Cloud Database` `base_context.md`

Observability: Langfuse Tracing Layer (100% Non-blocking)

AGENT 1: DEEP-DIVE

Instrumentation Agent: Deterministic & Robust Schema Synthesis

Key Engineering Decisions

1. LowCardinality & Type Inferences

Encodes repeating strings (e.g., `device_type`, `os`, `geoip_country_code`) into `LowCardinality(String)` to achieve massive compression and high-speed dictionary-based filtering.

2. Intelligent ORDER BY Keys

Rejects default arbitrary IDs in favor of high-cardinality query keys (e.g. `(destination, device_type, event_time)`) for fast range pruning.

3. Zero-Data-Loss Invariant

No `drop_table` primitive exists in code. Tables are widened via idempotent `ALTER TABLE ADD COLUMN` statements, ensuring existing production data is never lost.

Bounded Repair Loop Architecture

```
1. Profile → Deterministic stats extraction (no LLM cost)
2. Proposal → LLM outputs structured SchemaProposal
3. Lint & Dry-Run → Schema quality checks & Scratch DB execution
4. Execute & Reconcile → DDL applied to ClickHouse Cloud
```

Bounded by `DDL_REPAIR_ATTEMPTS` — prevents infinite loops and guarantees deterministic completion.

AGENT 2: DEEP-DIVE

Context Agent: Immutable Snapshot Layer in ClickHouse

Why ClickHouse for Context?

Rather than using external vector databases or unstable files on disk, PrismCH maintains the living business context layer inside ClickHouse itself across four specialized system tables:

`context_versions`
Full immutable snapshot tracking

`context_entries`
Structured metric & table formulas

`context_issues`
Contradictions & documentation gaps

`context_embeddings`
Fast semantic search index

Single Datastore Invariant: The Analytics Agent joins live data with business metrics in a single query engine—eliminating cross-system sync failures.

Automatic Cascade Trigger

When the Instrumentation Agent alters a schema, it notifies the Context Agent, which immediately merges schema changes, recalculates materialised view rationales, and triggers a fresh analytics run.

```
// Automatic Triggering Edge
Instrumentation Run Completed
  ↓ _notify_context()
Context Entry Merge & Snapshot
  ↓ _run_analytics()
Fresh Product Insights Generated
```

✓✓ Guarantees Analytics Agent NEVER operates on stale context!

AGENT 3: DEEP-DIVE

Analytics Agent: Server-Side Pushdown & Strict Safety

High-Performance Analytical SQL

The Analytics Agent generates native ClickHouse queries utilizing specialized high-speed aggregation functions:

`windowFunnel(...)`
Calculates conversion rates and step-by-step drop-offs across sliding temporal windows natively in ClickHouse.

`uniqCombined(...)`
HyperLogLog-based exact/approximate distinct counting for massive cardinality dimensions with minimal memory footprint.

3-Tier Query Guard Rails

Tier 1 **Client Whitelist Check (_is_destructive)**
Strict client-side check verifying statements start ONLY with `SELECT` or `WITH`.

Tier 2 **Raw SELECT Guard (_is_raw_select)**
Rejects `SELECT *` and non-aggregated queries. Forces ClickHouse to handle 100% of aggregations.

Tier 3 **Server-Side Enforcement (readonly=1)**
ClickHouse database session configured with `readonly=1` plus row, byte, and time caps.

OBSERVABILITY & AUDITING

End-to-End Traceability via Langfuse

Trace Architecture

Type-Enforced Mandatory Reasoning

Every agent step choice requires a `why` argument at the Python type level. An agent cannot record a decision without logging its rationale.

Context Version Binding

Every analytics step explicitly records the exact `context_version` used, enabling total historical auditability.

Non-Blocking Telemetry Safe-Guard

All tracing is wrapped in `_safe()` wrappers. A telemetry collector failure never disrupts the analytics pipeline.

Complete Audit Record

```
Root Trace: pipeline_run [ID: run_9f82a1]
  ├── agent_step: instrumentation.design_schema
  │   what: ORDER BY (destination, device_type)
  │   why: 90% predicates slice on destination
  ├── agent_step: context.refresh_schema
  │   context_version: v4
  └── agent_step: analytics.interpret_insights
      sql: SELECT windowFunnel(...) GROUP BY device_type
      cost: $0.00012 | tokens: 1,240
```

✓ Judge Trace Ready: every run prints its Langfuse trace ID.

COST & EFFICIENCY ARCHITECTURE

LLM Provider Choice & Token Optimization

Single-Interface Provider Dispatch

PrismCH features a clean abstraction layer (`prism_ch/agents/llm.py`) enabling instant provider swapping via `.env` without changing agent logic:

Gemini	Anthropic	OpenAI
`3.5-flash-lite`	`Claude`	`GPT`
Default (Fast/Cheap)	Supported	Supported

Why Gemini 3.5 Flash Lite? Priced far cheaper per token with sub-second execution speeds, ensuring fast repair loops and zero hackathon budget overruns.

Token Optimization Strategy

- **Deterministic Pre-Profiling:** Data typing and cardinalities are calculated programmatically in Python before hitting the LLM.
- **Server-Side Aggregations:** The LLM only interprets compact aggregated summary matrices (never raw row records).
- **Exact Cost Accounting:** Every trace tracks exact token usage and computes dollar cost in real time.

Per-model

Token pricing and USD cost on every LLM call

HACKATHON SPECIAL BENCHMARK

Built for the Unseen Spec (Spec #6)

Zero Hardcoding Guarantee

PrismCH contains **zero hardcoded rule sets** for the five known specs. The entire system operates purely on dynamic specification parsing and ClickHouse schema introspection.

Spec Ingestion Command	`make instrument SPEC=…`
Context Update	`Auto Cascaded`
Insight Report	`Generated with Trace ID`

Unseen Spec Execution Workflow

1. Ingests unseen `spec.md` and event sample NDJSON file.
2. Infers sorting keys and data types without human intervention.
3. Executes dry-run schema validation against scratch database.
4. Applies idempotent production DDL and loads NDJSON rows.
5. Context Agent automatically triggers full analysis report.

"No trace, no credit": Every artifact produced links directly to its Langfuse Trace ID.

CLICK-A-THON 2026

Judging Criteria Scorecard Summary

1. Schema Quality ✓ Covered

Optimal `LowCardinality` types, strategic `ORDER BY` pruning keys, and non-destructive schema widening.

2. Insight Quality ✓ Covered

Actionable PM reports explaining the "Why" behind drops across primary dimensions with confidence scores.

3. Context Freshness ✓ Covered

ClickHouse-resident immutable context layer with automated cascade triggers on schema changes.

4. Traceability ✓ Covered

Type-enforced decision reasoning, SQL pushdown logs, and token/cost accounting via Langfuse.

Atlys PrismCH is Ready for Deployment

Try the interactive web interface at `http://localhost:8000` or execute the full pipeline via CLI using `make run`.